

```

===== 文件: client4.0\config.js =====
/**
 * 下六儿 小游戏版 — 服务端地址配置
 * (与小程序版 client/miniprogram/config.js 保持一致)
 *
 * 环境切换:
 * - 开发 (微信开发者工具, 勾选"不校验合法域名") : USE_WSS = false
 * WS_BASE = 'ws://47.93.96.20:3000' (明文直连 IP, 工具可连)
 * - 生产 (真机体验版/正式版) : USE_WSS = true
 * 走 Nginx 反代: WS_BASE = 'wss://liuer.xin/ws'
 * 注意: 真机要求 wss:// + 已备案域名, 且需在微信公众平台
 * "服务器域名 → socket合法域名" 配置 wss://liuer.xin
 *
 * 切生产只需把 USE_WSS 改为 true (域名/证书就绪后) 。
 */
const PROD_DOMAIN = 'liuer.xin';
const USE_WSS = true; // 域名解析+ICP备案已完成, 走 wss://liuer.xin/ws
const SERVER_BASE = USE_WSS
? 'https://' + PROD_DOMAIN
: 'http://47.93.96.20:3000';
const WS_BASE = USE_WSS
? 'wss://' + PROD_DOMAIN + '/ws' // Nginx 反代路径 (见服务器 /etc/nginx/sites-available/game.conf)
: 'ws://47.93.96.20:3000';
// ===== 广告 / 分享恢复精力配置 =====
// 激励视频广告位 ID: 在微信公众平台「流量主→广告位管理」申请后填入, 未配置时走模拟播放
const AD_UNIT_ID = ''; // TODO: 填写你的激励视频广告位 ID
const AD_REWARD_DAILY = 3; // 看视频每日次数上限
const SHARE_REWARD_DAILY = 5; // 分享每日次数上限
module.exports = {
  SERVER_BASE, // HTTP 接口基址 (含 /api/auth/wx-login)
  WS_BASE, // WebSocket 基址
  USE_WSS,
  PROD_DOMAIN,
  AD_UNIT_ID,
  AD_REWARD_DAILY,
  SHARE_REWARD_DAILY,
};
===== 文件: client4.0\game.js =====
/**
 * 下六儿 小游戏版 — 入口 (game.js)
 *
 * 对应小程序版 app.js 的启动逻辑, 但小游戏没有 App() 入口,
 * 而是直接在 game.js 中执行:
 * 1. 初始化全局状态 + 登录 + 连接 WS
 * 2. 创建 Canvas 2D 上下文
 * 3. 注册所有场景并进入首页
 * 4. 启动 requestAnimationFrame 主循环, 每帧绘制当前场景

```

* 5. 监听触摸事件，转发给当前场景

*

* 【架构说明】

* 小游戏没有 WXML/页面路由，所有界面（首页/排行榜/我的/规则/对局）

* 都在同一块 Canvas 上用代码绘制；场景切换只是换一个绘制对象。

*/

```
const { state, init } = require('./state');
const { wsManager } = require('./utils/websocket');
const audio = require('./utils/audio');
const sceneMgr = require('./scenes/index');
// 注册场景
sceneMgr.register('home', require('./scenes/home'));
sceneMgr.register('rank', require('./scenes/rank'));
sceneMgr.register('profile', require('./scenes/profile'));
sceneMgr.register('rules', require('./scenes/rules'));
sceneMgr.register('match', require('./scenes/match'));
// 主画布上下文
let ctx = null;
let canvas = null;
let logicalW = 0; // 逻辑宽 (windowWidth)
let logicalH = 0; // 逻辑高 (windowHeight)
function main() {
  // 0. 读取启动参数 (分享卡片带 room 可自动进房)
  function readRoomFromLaunchOptions(opts) {
    if (opts && opts.query && opts.query.room) {
      state.pendingRoom = ('' + opts.query.room).toUpperCase().trim();
      console.log('[Game] 分享进入, pendingRoom=', state.pendingRoom);
    }
  }
  try {
    readRoomFromLaunchOptions((typeof wx.getLaunchOptionsSync === 'function') ? wx.getLaunchOptionsSync() : {});
  } catch (e) { /* ignore */ }
  // 热启动：好友从分享卡片点击进入已运行的小程序
  if (typeof wx.onShow === 'function') {
    wx.onShow((res) => readRoomFromLaunchOptions(res));
  }
  // 开启分享给好友 (小游戏)
  if (typeof wx.showShareMenu === 'function') {
    wx.showShareMenu({ withShareTicket: false, menus: ['shareAppMessage'] });
  }
  // 1. 登录并连接游戏服务器
  init();
}
```

```

// 2. 获取 Canvas
canvas = wx.createCanvas();
ctx = canvas.getContext('2d');
// 高清屏适配 (关键: 让画布按物理像素渲染, 避免文字/图形模糊)
// - canvas.width/height = 物理像素 (逻辑像素 × devicePixelRatio)
// - ctx.scale(dpr, dpr) 使后续绘制仍按逻辑坐标, 但以物理分辨率输出
const info = wx.getSystemInfoSync();
const dpr = info.pixelRatio || (info.devicePixelRatio || 1);
logicalW = info.windowWidth;
logicalH = info.windowHeight;
canvas.width = logicalW * dpr;
canvas.height = logicalH * dpr;
ctx.scale(dpr, dpr);
// 3. 进入首页
sceneMgr.goto('home');
// 4. 触摸事件转发
// 小游戏 wx.onTouchStart 的 Touch 对象坐标为 clientX/clientY (逻辑像素),
// 与 canvas.width/height (= windowWidth/windowHeight) 同一坐标系, 直接使用即可。
wx.onTouchStart((e) => {
  const t = e.touches[0];
  if (!t) return;
  const x = (t.x !== undefined) ? t.x : t.clientX;
  const y = (t.y !== undefined) ? t.y : t.clientY;
  // 首次交互后启动背景音乐 (小游戏要求用户手势后才能播放音频)
  audio.startBgm();
  sceneMgr.touch(x, y);
});
wx.onTouchMove((e) => {
  const t = e.touches[0];
  if (!t) return;
  const x = (t.x !== undefined) ? t.x : t.clientX;
  const y = (t.y !== undefined) ? t.y : t.clientY;
  sceneMgr.touchMove(x, y);
});
wx.onTouchEnd((e) => {
  sceneMgr.touchEnd();
});
// 5. 主循环 (各场景已在 onEnter 中自行注册 WS 监听)
loop();
}
function loop() {
  if (!ctx) return;
  // 清除按逻辑尺寸 (context 已 scale, 画布物理尺寸在 canvas.width/height)

```

```
ctx.clearRect(0, 0, logicalW, logicalH);
sceneMgr.draw(ctx);
requestAnimationFrame(loop);
}
// 启动
main();
===== 文件: client4.0\game.json =====
{
  "deviceOrientation": "portrait",
  "showStatusBar": true,
  "networkTimeout": {
    "request": 10000,
    "connectSocket": 10000,
    "uploadFile": 10000,
    "downloadFile": 10000
  },
  "subpackages": []
}
===== 文件: client4.0\project.config.json =====
{
  "appid": "wxccefff03956286b1",
  "compileType": "game",
  "projectname": "liuer4.0",
  "description": "下六儿 小游戏版（由小程序重构而来，满足游戏类目备案）",
  "setting": {
    "urlCheck": false,
    "es6": true,
    "enhance": true,
    "postcss": false,
    "minified": false,
    "newFeature": true,
    "compileWorklet": false,
    "uglifyFileName": false,
    "uploadWithSourceMap": true,
    "packNpmManually": false,
    "packNpmRelationList": [],
    "minifyWXSS": true,
    "minifyWXML": true,
    "localPlugins": false,
    "disableUseStrict": false,
    "useCompilerPlugins": false,
    "condition": false,
    "babelSetting": {
      "ignore": [],
      "disablePlugins": [],
      "outputPath": ""
    },
    "swc": false,
    "disableSWC": true
  }
}
```

```

},
"libVersion": "3.16.2",
"gameRoot": "./",
"condition": {},
"simulatorPluginLibVersion": {},
"packOptions": {
  "ignore": [],
  "include": []
},
"isGameTourist": false,
"editorSetting": {}
}
===== 文件: client4.0\project.private.config.json =====
{
  "projectname": "client4.0",
  "setting": {
    "compileHotReload": true,
    "urlCheck": true,
    "coverView": false,
    "lazyloadPlaceholderEnable": false,
    "skylineRenderEnable": false,
    "preloadBackgroundData": false,
    "autoAudits": false,
    "useApiHook": true,
    "showShadowRootInWxmlPanel": false,
    "useStaticServer": false,
    "useLanDebug": false,
    "showES6CompileOption": false,
    "checkInvalidKey": true,
    "ignoreDevUnusedFiles": true,
    "bigPackageSizeSupport": false
  },
  "libVersion": "3.16.2",
  "condition": {}
}
===== 文件: client4.0\scenes\home.js =====
/**
 * 下六儿 小游戏版 — 首页大厅场景
 * 设计依据: figma.md #page-home (375×812 基准, 暖金棕国风)
 *
 * 布局: 顶部资源栏 → 中央 (标题+棋盘动画+按钮) → 个人信息卡 (段位/积分/胜率/精力条) → 规则入口 → 底部导航
 * 用 Canvas 绘制; 好友开局弹出双 Tab 浮层 (复制房间号 / 进入房间)。
 */
const { wsManager } = require('../utils/websocket');
const { state, saveProfile, AVATAR_PRESETS, randomNickname } = require('../state');
const { PALETTE, drawButton, drawText, drawCard, drawAvatar, hit, roundRect, drawBottomNav } = require('../utils/ui');
const { SERVER_BASE } = require('../config');
const sceneMgr = require('./index');

```

```

let W = 375;
let H = 812;
let rects = {};
// 浮层状态
let overlay = null; // null | 'matching' | 'room' | 'energy' | 'profileSetup'
let overlayTab = 'create'; // 'create' 复制房间号 | 'join' 进入房间
let roomCode = ''; // 当前创建/加入的房间号
let myCreatedRoomCode = ''; // 自己创建的房间号 (用于拦截"进自己房间")
let joinInput = ''; // 进入房间输入的房间号
let joinError = '';
const inviters = []; // 收到的邀请 { roomId, nickName }
// 完善资料浮层状态
let profileNick = '';
let profileAvatar = 'emoji:' + AVATAR_PRESETS[0];
let profileAvatarIndex = 0;
let fightPhase = 0; // 棋盘打架动画相位
// ===== 生命周期 =====
function onEnter() {
  overlay = null;
  overlayTab = 'create';
  roomCode = '';
  joinInput = '';
  joinError = '';
  registerWs();
  // 首次进入且未设置资料: 弹出"完善资料"浮层
  if (state.showProfileSetup) {
    profileNick = randomNickname();
    profileAvatar = 'emoji:' + AVATAR_PRESETS[0];
    profileAvatarIndex = 0;
    overlay = 'profileSetup';
  }
  // 分享卡片带房间号 → 启动后自动进房
  handlePendingRoom();
}
function handlePendingRoom() {
  if (!state.pendingRoom) return;
  const code = state.pendingRoom;
  state.pendingRoom = '';
  if (!lwsManager.isConnected) {
    // 连接尚未就绪时延后重试 (登录/WS 连接异步)
    setTimeout(handlePendingRoom, 300);
  }
  state.pendingRoom = code;
}

```

```

return;
}
// 阻断: 不能进入自己创建的房间 (自己点自己分享的卡片)
if (myCreatedRoomCode && code.toUpperCase() === myCreatedRoomCode.toUpperCase()) {
  wx.showToast({ title: '不能进入自己创建的房间', icon: 'none' });
  return;
}
wsManager.send('join_room', { roomId: code });
overlay = 'matching';
}
function onLeave() {
  // 离开首页时清理 WS 监听, 避免 game_start 等跳转类监听残留导致重复触发场景切换
  removeWs();
}
function registerWs() {
  // 进入场景前先清掉旧监听, 避免重复注册导致一次 game_start 触发多次场景跳转
  removeWs();
  wsManager.on('match_status', (data) => {
    overlay = data.status === 'matching' ? 'matching' : (data.status === 'cancelled' ? null : overlay);
  });
  wsManager.on('game_start', (data) => {
    overlay = null;
    state.currentGame = data;
    sceneMgr.goto('match', data);
  });
  wsManager.on('room_created', (data) => {
    overlayTab = 'create';
    roomCode = data.roomId;
    myCreatedRoomCode = data.roomId;
  });
  wsManager.on('join_room_success', (data) => {
    overlayTab = 'create';
    roomCode = data.roomId;
  });
  wsManager.on('room_cancelled', () => {
    if (overlay === 'room') {
      wx.showToast({ title: '对方已离开房间', icon: 'none' });
      overlay = null;
      roomCode = "";
      joinInput = "";
      joinError = "";
    }
  });
  wsManager.on('room_expired', () => {
    if (overlay === 'inroom' && roomRole === 'creator') {
      wx.showToast({ title: '房间已过期, 请重新创建', icon: 'none' });
      overlay = null;
      roomCode = "";
    }
  });
}

```

```

roomRole = '';
}
});
wsManager.on('invite_received', (data) => {
  inviters.push({ roomId: data.roomId, nickName: data.nickName || '好友' });
});
wsManager.on('resource_update', (data) => {
  if (data.energy !== undefined) state.energy.current = data.energy;
  if (data.energyRecoverAt !== undefined) state.energy.nextRecoverAt = data.energyRecoverAt;
  if (data.energyMax !== undefined) state.energy.max = data.energyMax;
  if (data.rankScore !== undefined) state.rankScore = data.rankScore;
  if (data.rankName) state.rankName = data.rankName;
});
wsManager.on('sign_in_result', (data) => {
  if (data && data.energy !== undefined) state.energy.current = data.energy;
  const reward = data && data.bonus ? data.bonus : ((new Date().getDay() % 6 === 0) ? 10 : 5);
  wx.showToast({ title: '签到成功 +' + reward + '精力', icon: 'success' });
});
wsManager.on('ad_reward_result', (data) => {
  wx.showToast({ title: '精力 +' + ((data && data.reward) || 10), icon: 'success' });
});
wsManager.on('share_reward_result', (data) => {
  wx.showToast({ title: '精力 +' + ((data && data.reward) || 5), icon: 'success' });
});
wsManager.on('error', (data) => {
  const msg = data && data.errMsg ? data.errMsg : '';
  // 过滤良性/瞬时错误:
  // 1) 空 errMsg (服务器发的 error = {} 探针, 不影响游戏)
  // 2) 请先登录 (重连/命令早于登录完成时的竞态)
  // 3) 未知指令 (命令名拼写错误等开发期问题)
  if (!msg || msg === '请先登录' || /未知指令/.test(msg)) return;
  wx.showToast({ title: msg, icon: 'none' });
  if (overlay === 'matching' && /房间/.test(msg)) overlay = null;
});
}
function removeWs() {
  wsManager.off('match_status');
  wsManager.off('game_start');
  wsManager.off('room_created');
  wsManager.off('join_room_success');
  wsManager.off('room_cancelled');
  wsManager.off('room_expired');
  wsManager.off('invite_received');
  wsManager.off('resource_update');
  wsManager.off('error');
}
function onDraw(ctx) {
  // ctx.canvas.width 为物理尺寸, 需除以像素比得到逻辑尺寸

```

```

const pr = state.pixelRatio || 1;
W = ctx.canvas.width / pr;
H = ctx.canvas.height / pr;
// 首次进入且未设置资料：即刻弹出“完善资料”浮层（login 异步完成前 onEnter 可能未命中，
// 这里在每帧检测，保证一旦 showProfileSetup 置位立即引导，不先进入正常游戏）
if (state.showProfileSetup && !overlay) {
  profileNick = randomNickname();
  profileAvatar = 'emoji:' + AVATAR_PRESETS[0];
  profileAvatarIndex = 0;
  overlay = 'profileSetup';
}
// 热启动：分享卡片带房间号且当前在首页时，自动加入房间
handlePendingRoom();
drawBackground(ctx);
// 绘制顺序（自上而下）：
// 标题 + 棋盘 + slogan → 按钮 → 个人信息板 → 游戏规则
const btnBottom = drawCenter(ctx);
const navTop = H - 64; // 底部导航顶
rects.W = W; rects.H = H; // 先设置逻辑尺寸，供 drawBottomNav 使用
drawBottomNav(ctx, 'home', rects);
if (overlay === 'matching') drawMatchingOverlay(ctx);
else if (overlay === 'room') drawRoomOverlay(ctx);
else if (overlay === 'energy') drawEnergyOverlay(ctx);
else if (overlay === 'profileSetup') drawProfileSetupOverlay(ctx);
// 微信用户信息授权提示遮罩（授权浮层位于屏幕底部，这里仅作视觉提示）
if (state.authPending) drawAuthMask(ctx);
}
function drawAuthMask(ctx) {
  ctx.fillStyle = 'rgba(60,47,40,0.55)';
  ctx.fillRect(0, 0, W, H);
  const cy = state.statusBarHeight + 90;
  drawText(ctx, '请先授权微信昵称头像', W / 2, cy, { color: PALETTE.text, fontSize: 22, align: 'center', bold: true });
  drawText(ctx, '点击下方按钮完成授权后即可开始游戏', W / 2, cy + 32, { color: PALETTE.textDim, fontSize: 15, align: 'center' });
}
function drawBackground(ctx) {
  const g = ctx.createLinearGradient(0, 0, 0, H);
  g.addColorStop(0, PALETTE.bgGradientTop);
  g.addColorStop(1, PALETTE.bgGradientBottom);
  ctx.fillStyle = g;
  ctx.fillRect(0, 0, W, H);
}
// 顶部资源栏：仅显示下次恢复倒计时（精力值已在个人信息卡展示，避免重复）

```

```

function drawTopBar(ctx) {
  const y = state.statusBarHeight + 4;
  const right = W - 16;
  const recover = state.energy.nextRecoverAt > Date.now()
    ? formatCd(state.energy.nextRecoverAt)
    : '';
  if (!recover) return;
  drawText(ctx, '□ 下次+' + recover, right, y + 16, { color: PALETTE.textDim, fontSize: 13, align: 'right' });
}
// 中央：标题 → 棋盘 → slogan → 按钮 → 姓名板 → 游戏规则（整体垂直居中，间距均匀）
function drawCenter(ctx) {
  const cx = W / 2;
  const navTop = H - 64; // 底部导航顶
  const ruleH = 28; // 游戏规则行高
  const profileH = 120; // 姓名板高度
  const btnH = 46, btnW = W - 84, mx = (W - btnW) / 2;
  // 各区块之间的间距（控制疏密）
  const gapTitleBoard = 20; // 标题与棋盘间距
  const gapBoardSlogan = 20; // 棋盘与 slogan 间距
  const gapSloganBtn = 72; // slogan 与按钮间距
  const gapBtnProfile = 16; // 按钮与姓名板间距
  const gapProfileRule = 8; // 姓名板与游戏规则间距
  const gapRuleNav = 8; // 游戏规则与底部导航间距（= 姓名板-游戏规则间距）
  // 固定各区块高度
  const titleH = 46; // 标题行高（含大字）
  const sloganH = 20; // slogan 行高
  // 可用纵向范围：状态栏下方 ~ 底部导航上方
  const availTop = state.statusBarHeight + 8;
  const availBottom = navTop;
  const availH = availBottom - availTop;
  // 先计算除棋盘外的固定高度，反推棋盘尺寸（尽量大但留出上下留白）
  const fixedH = titleH + gapTitleBoard + gapBoardSlogan + sloganH
    + gapSloganBtn + (btnH * 2 + 12) + gapBtnProfile + profileH + gapProfileRule
    + ruleH + gapRuleNav;
  // 给棋盘留出边距（上标题下 slogan）。为了让按钮/姓名板/游戏规则明显下移，
  // 刻意把棋盘压小一些，把纵向空间让给下方的“按钮→姓名板→游戏规则”区块。
  // 棋盘边长 = 可用高 - 固定高 - 额外留白（这里的 extraTop 同时作为顶部下移量）
  const extraTop = 64 // ← 想下移更多就调大这个值（同时会压小棋盘）
  let boardH = Math.max(110, Math.min(200, availH - fixedH - extraTop));
  // 整个内容栈的总高度（含棋盘）
  let stackH = fixedH + boardH;
  // 若栈高超出可用区，压缩棋盘以确保不溢出（避免短屏被裁切）
  if (stackH > availH) {
    boardH = Math.max(100, boardH - (stackH - availH) - 8);
  }
}

```

```

stackH = fixedH + boardH;
}
// 栈顶 y: 从顶部固定下移 extraTop, 使按钮/姓名板/游戏规则整体下移
const stackTop = availTop + extraTop;
// 依序推算各区块 y
const titleBaseline = stackTop + titleH;
const boardY = stackTop + titleH + gapTitleBoard;
const sloganY = boardY + boardH + gapBoardSlogan;
const y1 = sloganY + sloganH + gapSloganBtn; // 第一按钮顶
const y2 = y1 + btnH + 12; // 第二按钮顶
const profileTop = y2 + btnH + gapBtnProfile; // 姓名板顶
const ruleY = profileTop + profileH + gapProfileRule + ruleH / 2; // 游戏规则基线
// === 标题 ===
drawTitlePlayful(ctx, cx, titleBaseline);
// === 棋盘卡 ===
const bx = (W - boardH) / 2;
drawBoardAnimationCard(ctx, bx, boardY, boardH);
// === Slogan ===
drawText(ctx, '落子布局·揪子博弈·走子决胜', cx, sloganY, { color: PALETTE.textDim, fontSize: 13, align: 'center' });
// === 按钮组 (在姓名板【上方】) ===
drawText(ctx, '消耗 5 点精力', cx, y1 - 7, { color: PALETTE.textDim, fontSize: 11, align: 'center' });
rects.match = drawButton(ctx, { text: '□ 随机匹配', x: mx, y: y1, w: btnW, h: btnH, fill: PALETTE.gold, textColor: PALETTE.textOnGold, fontSize: 21 });
rects.friend = drawButton(ctx, { text: '□ 邀请好友开局', x: mx, y: y2, w: btnW, h: btnH, fill: PALETTE.panel, textColor: PALETTE.gold, fontSize: 21, border: PALETTE.gold });
// === 姓名板 ===
drawProfileCard(ctx, profileTop);
// === 游戏规则 ===
drawRuleLink(ctx, ruleY);
return y2 + btnH;
}
/**
 * 标题"下六儿": 调皮字体
 * - "下六": 大字号 + 粗 + 卡通风 (圆体首选) + 主色棕
 * - "儿": 小字号, 靠在"六"的右下角略偏移
 */
function drawTitlePlayful(ctx, cx, y) {
// "下六" 大字 (圆体偏可爱, 整体棕褐色)
ctx.save();
const mainSize = 36;
const mainFont = '"Yuppy SC","Hiragino Maru Gothic ProN","Yuanti SC","Comic Sans MS","Marker Felt","PingFang SC","Microsoft YaHei",sans-serif';
const mainColor = PALETTE.text; // 统一深棕
ctx.font = `bold ${mainSize}px ${mainFont}`;
ctx.fillStyle = mainColor;

```

```

ctx.textAlign = 'center';
ctx.textBaseline = 'alphabetic';
const mainText = '下六';
const mainW = ctx.measureText(mainText).width;
ctx.fillText(mainText, cx, y);
// 用对齐"下六"的右侧来定位"儿", 确保不与"六"重叠
// "下六"以 cx 为中心, 右边缘 = cx + mainW/2
const sixRightEdge = cx + mainW / 2;
// "儿" 小字: 起点在"六"右边缘再往右 2px 处, 向下落到基线附近, 颜色统一深棕
const subSize = 18;
ctx.font = `bold ${subSize}px ${mainFont}`;
const subText = '儿';
ctx.textAlign = 'left';
const subX = sixRightEdge + 2;
const subY = y + 6;
ctx.fillStyle = mainColor;
ctx.fillText(subText, subX, subY);
ctx.restore();
}
/**
 * 棋盘卡 (figma 风格)
 * 外层: 白底圆角卡 (E8E3DA 描边)
 * 中层: 暖米色 #E8DBCF 棋盘底 (不超出边线, 但外层白底做一圈留白)
 * 网格: 深棕 #3C2F28, 1.5px 细线
 * 交叉点: 暖灰小圆 #C0B8A8
 * 棋子: 实色 (黑 #1A1A1A / 白 #FEFEFE + 灰描边)
 * 打架动画: 两子左右来回 ±12px
 */
function drawBoardAnimationCard(ctx, bx, by, bSize) {
  // 外卡: 白底 + 描边 (白边距留大一点, 让外圈白底清晰可见)
  drawCard(ctx, { x: bx, y: by, w: bSize, h: bSize, radius: 18, fill: 'FFFFFF', border: 'E8E3DA' });
  // 内棋盘: 暖米色填充 (缩进 12px, 确保外圈白底清晰)
  const pad = 14;
  const ix = bx + pad, iy = by + pad, iw = bSize - pad * 2, ih = bSize - pad * 2;
  roundRect(ctx, ix, iy, iw, ih, 10);
  ctx.fillStyle = 'E8DBCF';
  ctx.fill();
  // 网格区: 再向内缩进 10px, 让网格线不贴米色边线
  const grid = 5; // 5 条线分隔 6 格
  const gridPad = iw * 0.10;
  const gx = ix + gridPad, gy = iy + gridPad, gw = iw - gridPad * 2, gh = ih - gridPad * 2;
  ctx.strokeStyle = '#3C2F28';
  ctx.lineWidth = 1.5;
  for (let i = 0; i <= grid; i++) {
    const p = (gw / grid) * i;
    ctx.beginPath();
    ctx.moveTo(gx + p, gy);
    ctx.lineTo(gx + p, gy + gh);
  }

```

```

ctx.stroke();
ctx.beginPath();
ctx.moveTo(gx, gy + p);
ctx.lineTo(gx + gw, gy + p);
ctx.stroke();
}
// 交叉点小圆
ctx.fillStyle = '#C0B8A8';
for (let r = 0; r <= grid; r++) {
  for (let c = 0; c <= grid; c++) {
    ctx.beginPath();
    ctx.arc(gx + (gw / grid) * c, gy + (gh / grid) * r, 1.8, 0, Math.PI * 2);
    ctx.fill();
  }
}
// 打架棋子：实色（位于网格区中心偏上）
const step = gw / grid;
const off = Math.sin(Date.now() / 420) * 12;
const r1 = Math.max(6, step * 0.34);
drawSolidPiece(ctx, gx + 2 * step + off, gy + 2 * step, r1, '#1A1A1A', null);
drawSolidPiece(ctx, gx + 3 * step - off, gy + 3 * step, r1, '#FEFEFE', '#D0D0D0');
}
/** 绘制实色棋子（figma 风格：轻微阴影 + 不透明填充 + 描边） */
function drawSolidPiece(ctx, x, y, r, fill, stroke) {
  // 轻微阴影（落地感）
  ctx.beginPath();
  ctx.arc(x, y + 1.5, r, 0, Math.PI * 2);
  ctx.fillStyle = 'rgba(60,47,40,0.22)';
  ctx.fill();
  // 主体
  ctx.beginPath();
  ctx.arc(x, y, r, 0, Math.PI * 2);
  ctx.fillStyle = fill;
  ctx.fill();
  if (stroke) {
    ctx.lineWidth = 1.5;
    ctx.strokeStyle = stroke;
    ctx.stroke();
  }
}
// 个人信息卡（传入顶部 y，高度固定 120，参照 figma 首页个人信息卡）
function drawProfileCard(ctx, cardY) {
  const cardX = 16;
  const cardW = W - 32;
  const cardH = 120;
  drawCard(ctx, { x: cardX, y: cardY, w: cardW, h: cardH, radius: 16 });
}

```

```

const cy = cardY + 30;
drawAvatar(ctx, { x: cardX + 34, y: cy, r: 22, label: (state.userInfo && state.userInfo.nickName) || '我', avatar: (state.userInfo && state.userInfo.avatarUrl) || '', ring: true
});
drawText(ctx, (state.userInfo && state.userInfo.nickName) || '我', cardX + 70, cy - 8, { color: PALETTE.text, fontSize: 19, bold: true });
// 段位徽章
const badgeW = 74;
const badgeX = cardX + 70;
const badgeY = cy + 8;
roundRect(ctx, badgeX, badgeY, badgeW, 22, 11);
ctx.fillStyle = PALETTE.goldBright;
ctx.fill();
drawText(ctx, state.rankName || '初级小六', badgeX + badgeW / 2, badgeY + 15, { color: PALETTE.textOnGold, fontSize: 12, align: 'center', bold: true });
// 积分 / 胜率 (胜率最多保留一位小数, 字号缩小避免溢出卡片右边界)
const winRateText = (Number(state.winRate) || 0).toFixed(1) + '%';
drawText(ctx, " + (state.rankScore || 0), cardX + cardW - 88, cy - 4, { color: PALETTE.gold, fontSize: 20, align: 'center', bold: true });
drawText(ctx, '积分', cardX + cardW - 88, cy + 16, { color: PALETTE.textDim, fontSize: 11, align: 'center' });
drawText(ctx, winRateText, cardX + cardW - 30, cy - 4, { color: PALETTE.green, fontSize: 17, align: 'center', bold: true });
drawText(ctx, '胜率', cardX + cardW - 30, cy + 16, { color: PALETTE.textDim, fontSize: 11, align: 'center' });
// 精力进度条
const barX = cardX + 16;
const barY = cardY + 68;
const barW = cardW - 32;
roundRect(ctx, barX, barY, barW, 8, 4);
ctx.fillStyle = PALETTE.panelBorder;
ctx.fill();
const pct = Math.max(0, Math.min(1, state.energy.current / state.energy.max));
if (pct > 0) {
  roundRect(ctx, barX, barY, barW * pct, 8, 4);
  ctx.fillStyle = PALETTE.green;
  ctx.fill();
}
// 精力数字与下次恢复时间放在精力条【下方】，左右分列（避免与上方头像重叠）
drawText(ctx, '精力' + state.energy.current + '/' + state.energy.max, barX, barY + 22, { color: PALETTE.text, fontSize: 12, baseline: 'middle' });
drawText(ctx, '下次恢复 □' + formatCd(state.energy.nextRecoverAt), cardX + cardW - 16, barY + 22, { color: PALETTE.textDim, fontSize: 11, align: 'right', baseline: 'middle' });
}
function formatCd(ts) {
  if (!ts) return '00:00';
  const s = Math.max(0, Math.floor((ts - Date.now()) / 1000));
  const m = Math.floor(s / 60);
  const ss = s % 60;
  return (m < 10 ? '0' : '') + m + ':' + (ss < 10 ? '0' : '') + ss;
}
function drawRuleLink(ctx, y) {
  // 点击区域：限制在底部导航【之上】，避免与底部导航重叠导致误触
  rects.ruleLink = { x: 0, y: y - 12, w: W, h: 24 };
  drawText(ctx, '游戏规则 □', W / 2, y + 6, { color: PALETTE.textDim, fontSize: 15, align: 'center' });
}

```

```

}
// ===== 匹配浮层 =====
function drawMatchingOverlay(ctx) {
  dim(ctx);
  const pw = W * 0.8, ph = Math.max(220, Math.round(H * 0.36)), px = (W - pw) / 2, py = (H - ph) / 2;
  drawCard(ctx, { x: px, y: py, w: pw, h: ph, radius: 24 });
  drawText(ctx, '匹配中...', W / 2, py + 70, { color: PALETTE.text, fontSize: 32, align: 'center', bold: true });
  drawText(ctx, '正在为你寻找对手', W / 2, py + 120, { color: PALETTE.textDim, fontSize: 20, align: 'center' });
  rects.cancelMatch = drawButton(ctx, { text: '取消匹配', x: px + 40, y: py + ph - 80, w: pw - 80, h: 56, fill: PALETTE.red, textColor: 'FFFFFF', fontSize: 24 });
}
// ===== 好友开局浮层 (双 Tab) =====
function drawRoomOverlay(ctx) {
  dim(ctx);
  const pw = W * 0.86, ph = Math.max(360, Math.round(H * 0.58)), px = (W - pw) / 2, py = (H - ph) / 2;
  drawCard(ctx, { x: px, y: py, w: pw, h: ph, radius: 24 });
  // 关闭按钮
  rects.closeRoom = { x: px + pw - 40, y: py + 12, w: 28, h: 28 };
  drawText(ctx, '□', px + pw - 26, py + 32, { color: PALETTE.textDim, fontSize: 22, align: 'center' });
  drawText(ctx, '好友开局', W / 2, py + 44, { color: PALETTE.text, fontSize: 26, align: 'center', bold: true });
  // Tab 切换
  const tabY = py + 64;
  const tabW = (pw - 48) / 2;
  const tabX1 = px + 24;
  const tabX2 = px + 24 + tabW + 0;
  rects.tabCreate = drawButton(ctx, { text: '复制房间号', x: tabX1, y: tabY, w: tabW, h: 42, fill: overlayTab === 'create' ? PALETTE.gold : PALETTE.panel, textColor: overlayTab === 'create' ? PALETTE.textOnGold : PALETTE.textDim, fontSize: 18, border: overlayTab === 'create' ? null : PALETTE.panelBorder });
  rects.tabJoin = drawButton(ctx, { text: '进入房间', x: tabX2, y: tabY, w: tabW, h: 42, fill: overlayTab === 'join' ? PALETTE.gold : PALETTE.panel, textColor: overlayTab === 'join' ? PALETTE.textOnGold : PALETTE.textDim, fontSize: 18, border: overlayTab === 'join' ? null : PALETTE.panelBorder });
  if (overlayTab === 'create') {
    drawCreateTab(ctx, px, py, pw);
  } else {
    drawJoinTab(ctx, px, py, pw);
  }
}
function drawCreateTab(ctx, px, py, pw) {
  const y = py + 130;
  if (!roomCode) {
    rects.doCreate = drawButton(ctx, { text: '创建房间', x: px + 40, y: y, w: pw - 80, h: 54, fill: PALETTE.gold, textColor: PALETTE.textOnGold, fontSize: 24 });
    drawText(ctx, '创建后分享房间号给好友', W / 2, y + 92, { color: PALETTE.textDim, fontSize: 16, align: 'center' });
    return;
  }
}
// 房间号展示
drawText(ctx, '房间号', W / 2, y + 10, { color: PALETTE.textDim, fontSize: 18, align: 'center' });
drawText(ctx, roomCode, W / 2, y + 52, { color: PALETTE.gold, fontSize: 44, align: 'center', bold: true });
rects.copyRoom = drawButton(ctx, { text: '复制房间号', x: px + 40, y: y + 78, w: pw - 80, h: 52, fill: PALETTE.gold, textColor: PALETTE.textOnGold, fontSize: 22 });

```

```

rects.shareRoom = drawButton(ctx, { text: '分享房间号给好友', x: px + 40, y: y + 142, w: pw - 80, h: 52, fill: PALETTE.panel, textColor: PALETTE.green, fontSize: 22,
border: PALETTE.green });
}
function drawJoinTab(ctx, px, py, pw) {
const y = py + 130;
drawText(ctx, '输入好友分享的房间号', W / 2, y + 6, { color: PALETTE.textDim, fontSize: 16, align: 'center' });
// 输入框 (模拟)
const iw = pw - 80, ih = 52, ix = px + 40, iy = y + 22;
roundRect(ctx, ix, iy, iw, ih, 12);
ctx.fillStyle = PALETTE.bg;
ctx.fill();
ctx.lineWidth = 1.5;
ctx.strokeStyle = PALETTE.panelBorder;
ctx.stroke();
rects.joinInput = { x: ix, y: iy, w: iw, h: ih };
const placeholder = joinInput || '例如 ABC123';
drawText(ctx, placeholder, ix + iw / 2, iy + ih / 2 + 6, { color: joinInput ? PALETTE.text : PALETTE.textDim, fontSize: 24, align: 'center', bold: !!joinInput });
if (joinError) drawText(ctx, joinError, W / 2, iy + ih + 24, { color: PALETTE.red, fontSize: 14, align: 'center' });
rects.doJoin = drawButton(ctx, { text: '进入房间', x: px + 40, y: iy + ih + 36, w: pw - 80, h: 52, fill: PALETTE.gold, textColor: PALETTE.textOnGold, fontSize: 22 });
}
function dim(ctx) {
ctx.fillStyle = 'rgba(60,47,40,0.5)';
ctx.fillRect(0, 0, W, H);
}
// ===== 触摸 =====
function onTouch(x, y) {
// 微信用户信息授权浮层显示期间, 禁用所有其它按钮, 避免与授权按钮重叠触发
if (state.authPending) return;
// 底部导航优先判定 (避免被"游戏规则"扩大区域误触)
if (rects.bottomTabs) {
for (const t of rects.bottomTabs) {
if (hit(t, x, y) && t.key !== 'home') { sceneMgr.goto(t.key); return; }
}
}
if (overlay === 'matching') {
if (hit(rects.cancelMatch, x, y)) { wsManager.send('match_cancel'); overlay = null; }
return;
}
if (overlay === 'room') { handleRoomTouch(x, y); return; }
if (overlay === 'energy') { handleEnergyTouch(x, y); return; }
if (overlay === 'profileSetup') { handleProfileSetupTouch(x, y); return; }
if (hit(rects.match, x, y)) { startMatch(); return; }
if (hit(rects.friend, x, y)) { openInvite(); return; }
if (hit(rects.ruleLink, x, y)) { sceneMgr.goto('rules'); return; }
}

```

```

function handleRoomTouch(x, y) {
  if (hit(rects.closeRoom, x, y)) { overlay = null; return; }
  if (hit(rects.tabCreate, x, y)) { overlayTab = 'create'; return; }
  if (hit(rects.tabJoin, x, y)) { overlayTab = 'join'; return; }
  if (overlayTab === 'create') {
    if (!roomCode) {
      if (hit(rects.doCreate, x, y)) { wsManager.send('invite_room'); }
      return;
    }
    if (hit(rects.copyRoom, x, y)) {
      wx.showToast({ title: '复制中...', icon: 'loading' });
      wx.setClipboardData({
        data: roomCode,
        success: () => {
          wx.showToast({ title: '房间号已复制', icon: 'success' });
        },
        fail: (err) => {
          console.error('[Home] 复制房间号失败:', err);
          wx.showModal({ title: '复制失败', content: '请手动复制房间号: ' + roomCode, showCancel: false });
        },
      });
    } else if (hit(rects.shareRoom, x, y)) {
      shareRoom();
    }
    return;
  }
  if (overlayTab === 'join') {
    if (hit(rects.joinInput, x, y)) {
      wx.showKeyboard ? null : null;
      wx.showModal({
        title: '输入房间号',
        editable: true,
        placeholderText: '例如 ABC123',
        success: (r) => {
          if (r.confirm && r.content) { joinInput = (r.content || '').toUpperCase().trim(); joinError = ''; }
        },
      });
      return;
    }
    if (hit(rects.doJoin, x, y)) {
      const code = joinInput.trim();
      if (!code) { joinError = '请输入房间号'; return; }
      // 阻断：不能进入自己创建的房间（否则双方为同一人，游戏异常）
      if (myCreatedRoomCode && code.toUpperCase() === myCreatedRoomCode.toUpperCase()) {
        joinError = '不能进入自己创建的房间';
        wx.showToast({ title: '不能进入自己创建的房间', icon: 'none' });
        return;
      }
    }
  }
}

```

```

}
wsManager.send('join_room', { roomId: code });
// 加入成功后服务端会自动 game_start, 浮层由 game_start 事件关闭
}
}
function leaveRoom() {
wsManager.send('leave_room', { roomId: roomCode });
overlay = null;
roomCode = '';
joinInput = '';
joinError = '';
}
function startMatch() {
if (state.energy.current < 5) { overlay = 'energy'; return; }
wsManager.send('match_start');
overlay = 'matching';
}
function openInvite() {
if (state.energy.current < 5) { overlay = 'energy'; return; }
overlay = 'room';
overlayTab = 'create';
if (!roomCode) wsManager.send('invite_room');
}
function shareRoom() {
if (!roomCode) return;
// 小游戏分享给好友, 带 query.room, 好友点开可自动进房
wx.shareAppMessage && wx.shareAppMessage({
title: '【下六儿】邀你来对战! 房间号' + roomCode,
imageUrl: '',
query: { room: roomCode },
success: () => wx.showToast({ title: '已唤起分享', icon: 'success' }),
fail: () => wx.showToast({ title: '分享取消', icon: 'none' }),
});
}
// ===== 精力不足恢复浮层 =====
function drawEnergyOverlay(ctx) {
dim(ctx);
const pw = W * 0.86, ph = Math.max(400, Math.round(H * 0.62)), px = (W - pw) / 2, py = (H - ph) / 2;
drawCard(ctx, { x: px, y: py, w: pw, h: ph, radius: 24 });
rects.closeEnergy = { x: px + pw - 40, y: py + 12, w: 28, h: 28 };
drawText(ctx, '□', px + pw - 26, py + 32, { color: PALETTE.textDim, fontSize: 22, align: 'center' });
drawText(ctx, '精力不足', W / 2, py + 48, { color: PALETTE.text, fontSize: 28, align: 'center', bold: true });
}

```

```

drawText(ctx, '每局消耗 5 点 · 每 5 分钟自动恢复 1 点 (离线也计)', W / 2, py + 82, { color: PALETTE.textDim, fontSize: 14, align: 'center' });
const bx = px + 40, bw = pw - 80, by = py + 116, bh = 52, gap = 14;
rects.adEnergy = drawButton(ctx, { text: '👁 看广告 +10 精力', x: bx, y: by, w: bw, h: bh, fill: PALETTE.gold, textColor: PALETTE.textOnGold, fontSize: 19 });
rects.shareEnergy = drawButton(ctx, { text: '📢 分享 +5 精力', x: bx, y: by + (bh + gap), w: bw, h: bh, fill: PALETTE.green, textColor: 'FFFFFF', fontSize: 19 });
rects.signEnergy = drawButton(ctx, { text: '📅 每日签到 +5', x: bx, y: by + (bh + gap) * 2, w: bw, h: bh, fill: PALETTE.panel, textColor: PALETTE.gold, fontSize: 19,
border: PALETTE.gold });
drawText(ctx, '也可等待自然恢复, 关闭后继续', W / 2, py + ph - 26, { color: PALETTE.textDim, fontSize: 13, align: 'center' });
}
function handleEnergyTouch(x, y) {
if (hit(rects.closeEnergy, x, y)) { overlay = null; return; }
if (hit(rects.adEnergy, x, y)) {
wx.showLoading() && wx.showLoading({ title: '获取中', mask: true });
wsManager.send('get_ad_reward');
setTimeout(() => wx.hideLoading() && wx.hideLoading(), 800);
overlay = null;
return;
}
if (hit(rects.shareEnergy, x, y)) {
wsManager.send('get_share_reward');
wx.showToast({ title: '已获得精力 +5', icon: 'success' });
overlay = null;
return;
}
if (hit(rects.signEnergy, x, y)) {
wx.showLoading() && wx.showLoading({ title: '签到中', mask: true });
wsManager.send('sign_in');
setTimeout(() => wx.hideLoading() && wx.hideLoading(), 800);
overlay = null;
return;
}
}
// ===== 完善资料浮层 (引导设置昵称 + 头像) =====
function drawProfileSetupOverlay(ctx) {
dim(ctx);
const pw = W * 0.88, ph = Math.max(440, Math.round(H * 0.68)), px = (W - pw) / 2, py = (H - ph) / 2;
drawCard(ctx, { x: px, y: py, w: pw, h: ph, radius: 24 });
drawText(ctx, '完善资料', W / 2, py + 44, { color: PALETTE.text, fontSize: 28, align: 'center', bold: true });
// 当前选中的头像 (大圆)
const bigR = 46;
drawAvatar(ctx, { x: W / 2, y: py + 110, r: bigR, avatar: profileAvatar, label: profileNick.slice(0, 1), ring: true });
// 预设头像排 (6 个 emoji)
const avatarR = 26;
const gap = (pw - 48 - avatarR * 2 * 3) / 2; // 每行3个
const startX = px + 24 + avatarR;

```

```

const rowY = py + 175;
rects.presetAvatars = [];
AVATAR_PRESETS.forEach((emoji, i) => {
  const ax = startX + (i % 3) * ((avatarR * 2 + gap));
  const ay = rowY + Math.floor(i / 3) * (avatarR * 2 + 14);
  const selected = i === profileAvatarIndex;
  ctx.beginPath();
  ctx.arc(ax, ay, avatarR + 3, 0, Math.PI * 2);
  ctx.fillStyle = selected ? PALETTE.gold : '#F0E9DB';
  ctx.fill();
  drawAvatar(ctx, { x: ax, y: ay, r: avatarR, avatar: 'emoji:' + emoji, label: '' });
  rects.presetAvatars.push({ x: ax - avatarR - 3, y: ay - avatarR - 3, w: avatarR * 2 + 6, h: avatarR * 2 + 6, emoji });
});
// 上传相册按钮
rects.uploadAvatar = drawButton(ctx, {
  text: '上传相册图片', x: px + 40, y: py + 252, w: pw - 80, h: 44,
  fill: PALETTE.panel, textColor: PALETTE.green, fontSize: 18, border: PALETTE.green,
});
// 昵称区
drawText(ctx, '我的昵称', px + 24, py + 318, { color: PALETTE.textDim, fontSize: 16 });
drawText(ctx, profileNick || '未设置', px + 24, py + 352, { color: PALETTE.text, fontSize: 24, bold: true });
rects.nickShuffle = drawButton(ctx, {
  text: '换一个', x: px + pw - 180, y: py + 326, w: 80, h: 38,
  fill: PALETTE.panel, textColor: PALETTE.gold, fontSize: 16, border: PALETTE.gold,
});
rects.nickEdit = drawButton(ctx, {
  text: '输入昵称', x: px + pw - 92, y: py + 326, w: 72, h: 38,
  fill: PALETTE.gold, textColor: PALETTE.textOnGold, fontSize: 16,
});
// 确定
rects.profileConfirm = drawButton(ctx, {
  text: '开始游戏', x: px + 40, y: py + ph - 64, w: pw - 80, h: 48,
  fill: PALETTE.gold, textColor: PALETTE.textOnGold, fontSize: 22,
});
}
function handleProfileSetupTouch(x, y) {
  // 预设头像选择
  if (rects.presetAvatars) {
    for (let i = 0; i < rects.presetAvatars.length; i++) {
      if (hit(rects.presetAvatars[i], x, y)) {
        profileAvatarIndex = i;
        profileAvatar = 'emoji:' + AVATAR_PRESETS[i];
        return;
      }
    }
  }
}

```

```

// 上传相册图片
if (hit(rects.uploadAvatar, x, y)) { uploadAvatar(); return; }
// 随机昵称
if (hit(rects.nickShuffle, x, y)) { profileNick = randomNickname(); return; }
// 输入昵称
if (hit(rects.nickEdit, x, y)) {
  wx.showModal({
    title: '设置昵称',
    editable: true,
    placeholderText: '请输入昵称 (最多10字)',
    success: (r) => { if (r.confirm && r.content && r.content.trim()) profileNick = r.content.trim().slice(0, 10); },
  });
  return;
}
// 确定
if (hit(rects.profileConfirm, x, y)) {
  if (!profileNick.trim()) { wx.showToast({ title: '请先设置昵称', icon: 'none' }); return; }
  saveProfile(profileNick.trim(), profileAvatar);
  wsManager.send('update_profile', { nickName: profileNick.trim(), avatarUrl: profileAvatar });
  overlay = null;
  return;
}
}
/** 选择相册图片并上传, 成功后作为头像 */
function uploadAvatar() {
  // 小游戏环境不支持 wx.chooseImage, 需用 wx.chooseMedia
  wx.chooseMedia({
    count: 1,
    mediaType: ['image'],
    sizeType: ['compressed'],
    sourceType: ['album'],
    success: (res) => {
      const file = res.tempFiles && res.tempFiles[0];
      const tempPath = file && file.tempFilePath;
      if (!tempPath) return;
      wx.showLoading({ title: '上传中', mask: true });
      // 转 base64
      wx.getFileSystemManager().readFile({
        filePath: tempPath,
        encoding: 'base64',
        success: (fr) => {
          wx.request({
            url: SERVER_BASE + '/api/avatar/upload',
            method: 'POST',
            header: { 'Content-Type': 'application/json' },
            data: { openid: state.openid || '', base64: fr.data || '' },
            success: (rr) => {
              wx.hideLoading();
              if (rr.statusCode === 200 && rr.data && rr.data.ok && rr.data.url) {

```

```

profileAvatar = rr.data.url;
profileAvatarIndex = -1;
wx.showToast({ title: '头像已更新', icon: 'success' });
} else {
wx.showToast({ title: '上传失败', icon: 'none' });
}
},
fail: () => { wx.hideLoading(); wx.showToast({ title: '上传失败', icon: 'none' }); },
});
},
fail: () => { wx.hideLoading(); wx.showToast({ title: '读取图片失败', icon: 'none' }); },
});
},
fail: (err) => { wx.showToast({ title: '取消选择', icon: 'none' }); },
});
}
module.exports = { onEnter, onLeave, onDraw, onTouch, onWs: () => {} };
===== 文件: client4.0\scenes\index.js =====
/**
 * 下六儿 小游戏版 — 场景管理器
 *
 * 小游戏没有页面路由 (wx.navigateTo) , 所有界面都在同一块 Canvas 上绘制。
 * 这里用一个简单的 "当前场景" 切换机制:
 * - 每个场景是一个对象, 提供 onEnter / onDraw(ctx) / onTouch(x,y) / onWs(cmd,data)
 * - game.js 主循环每帧调用 当前场景.onDraw
 * - 触摸事件转发给当前场景.onTouch
 *
 * 场景列表:
 * home 首页大厅
 * rank 排行榜
 * profile 我的
 * rules 规则说明
 * match 对局 (含 6x6 棋盘)
 * (overlay) 匹配中弹窗等以 home 内的浮层实现
 */
const scenes = {};
let current = null;
let currentName = "";
function register(name, scene) {
scenes[name] = scene;
}
function goto(name, payload) {
if (!scenes[name]) {
console.error('[Scene] 未注册场景:', name);
return;
}
}

```

```
if (current && current.onLeave) current.onLeave();
current = scenes[name];
currentName = name;
if (current.onEnter) current.onEnter(payload || {});
console.log('[Scene] 切换到:', name);
}
function getCurrent() {
return current;
}
function getCurrentName() {
return currentName;
}
/** 主循环每帧调用 */
function draw(ctx) {
if (current && current.onDraw) current.onDraw(ctx);
}
/** 触摸事件转发 */
function touch(x, y) {
if (current && current.onTouch) current.onTouch(x, y);
}
/** 触摸移动转发（用于列表拖动等） */
function touchMove(x, y) {
if (current && current.onTouchMove) current.onTouchMove(x, y);
}
/** 触摸结束转发 */
function touchEnd() {
if (current && current.onTouchEnd) current.onTouchEnd();
}
/** WebSocket 消息转发 */
function dispatchWs(cmd, data) {
if (current && current.onWs) current.onWs(cmd, data);
}
module.exports = {
register,
goto,
getCurrent,
getCurrentName,
draw,
touch,
touchMove,
touchEnd,
dispatchWs,
```

```

};
===== 文件: client4.0\scenes\match.js =====
/**
 * 下六儿 小游戏版 — 对局场景
 * (对应小程序版 pages/match/match, UI 由 WXML 改为 Canvas 绘制)
 * 设计风格: 暖金棕国风 (figma 设计稿) 。
 */
const { wsManager } = require('./utils/websocket');
const { state } = require('./state');
const ui = require('./utils/ui');
const { PALETTE, drawText, drawButton, drawCard, drawAvatar, hit, roundRect, PIECE_SKINS } = ui;
const gameCore = require('./utils/game-core');
const audio = require('./utils/audio');
const settingsModal = require('./utils/settings-modal');
const sceneMgr = require('./index');
let W = 375;
let H = 667;
let rects = {};
const game = {
  gameId: '', phase: 'place', phaseLabel: '下子阶段', currentTurn: 0,
  board: [], remainingTime: 0, timeTotal: 15, myColor: 0, myOpenid: '',
  myInfo: {}, opponentInfo: {}, myTurn: false,
  myCatchNum: 0, opponentCatchNum: 0, catchNums: { black: 0, white: 0 },
  myRemainText: '18/18', opponentRemainText: '18/18',
  moveCaptureMode: false, boardPieces: [], legalCells: [], selectedPieceIndex: -1,
  skipAvailable: false, drawPaused: false, showSettle: false, settleData: {},
  myResult: '', scoreChange: 0, drawRequestBy: '', drawRequestName: '',
  showDrawRequest: false, showRequestDraw: false, showGiveUpConfirm: false, showSettings: false, boardFlipped: false,
  timerText: '00:00', timeLow: false, timerProgress: 100,
};
let boardGeo = { ox: 0, oy: 0, step: 0, size: 0 };
// 本地倒计时 tick: 用本地时间递减, 保证秒数字每秒变化 (不等服务端推)
let lastTickTs = 0;
// 各阶段配色 (下子蓝/揪子红/走子绿): 每项含主色 main、深色 dark (渐变/阴影用)
const PHASE_COLORS = {
  place: { main: '#3B82C4', dark: '#2A5F96' }, // 下子-蓝 (高级靛蓝)
  capture: { main: '#D5484A', dark: '#A9383A' }, // 揪子-红 (朱红)
  move: { main: '#3FA768', dark: '#2C7D4E' }, // 走子-绿 (青竹绿)
  settled: { main: PALETTE.gold, dark: '#6B5210' },
};
/** 获取当前阶段的配色对象 */
function phaseColor() {
  return PHASE_COLORS[game.phase] || PHASE_COLORS.settled;
}

```

```

function calcTimerProgress(remainingTime) {
  const total = game.timeTotal || 15;
  const t = Math.max(0, Math.min(total, remainingTime || 0));
  return Math.round((t / total) * 100);
}
function onEnter(payload) {
  const gameData = (payload && payload.gameld) ? payload : state.currentGame;
  // 进入新局前，先清除旧对局残留的 WS 监听器，避免收到旧消息
  removeWs();
  if (!gameData) {
    wx.showToast({ title: '对局数据丢失', icon: 'none' });
    setTimeout(() => sceneMgr.goto('home'), 1500);
    return;
  }
  const myOpenid = state.openid;
  const myColor = gameData.blackPlayer.openid === myOpenid ? 1 : 2;
  const opponentColor = myColor === 1 ? 2 : 1;
  const myInfo = myColor === 1 ? gameData.blackPlayer : gameData.whitePlayer;
  const oppInfo = opponentColor === 1 ? gameData.blackPlayer : gameData.whitePlayer;
  const boardFlipped = myColor === 2;
  const myTurn = gameData.currentTurn === myColor;
  const timeTotalSec = Math.ceil((gameData.timeLimit || 15000) / 1000); // 统一为秒
  const remainingTime = gameData.remainingTime || timeTotalSec;
  const phase = gameCore.resolveStage(gameData.stage);
  Object.assign(game, {
    gameld: gameData.gameld, phase,
    phaseLabel: gameCore.STAGE_LABELS[phase] || '下子阶段',
    currentTurn: gameData.currentTurn, myColor, myOpenid,
    myInfo: { nickName: myInfo.nickName || '', avatarUrl: myInfo.avatarUrl || '', rankScore: myInfo.rankScore || 0 },
    opponentInfo: { nickName: oppInfo.nickName || '', avatarUrl: oppInfo.avatarUrl || '', rankScore: oppInfo.rankScore || 0 },
    myTurn, boardFlipped, remainingTime, timeTotal: timeTotalSec,
    timerText: gameCore.formatTime(remainingTime),
    timeLow: remainingTime > 0 && remainingTime <= 10,
    timerProgress: calcTimerProgress(remainingTime),
    catchNums: { black: 0, white: 0 }, board: [], boardPieces: [],
    legalCells: [], selectedPieceIndex: -1, skipAvailable: false,
    showSettle: false, drawPaused: false, showDrawRequest: false, showRequestDraw: false, showGiveUpConfirm: false, showSettings: false,
  });
  updateBoardPieces(gameData.board || []);
  updateCatchNums(gameData.catchNums || { black: 0, white: 0 });
  registerWs();
  state.currentGame = null;
}
function onLeave() {
  // 如果用户主动离开对局且尚未结算，自动判负（避免对手无限等待）

```

```

if (game.phase !== 'settled' && game.gameld) {
  wsManager.send('give_up');
}
removeWs();
}
// ===== WebSocket =====
function registerWs() {
  wsManager.on('stage_change', onStageChange);
  wsManager.on('piece_placed', onBoardUpdate);
  wsManager.on('capture_made', onBoardUpdate);
  wsManager.on('move_made', onBoardUpdate);
  wsManager.on('linked_capture', onLinkedCapture);
  wsManager.on('draw_rejected', onDrawRejected);
  wsManager.on('draw_request', onDrawRequest);
  wsManager.on('game_settle', onGameSettle);
  wsManager.on('game_snapshot', onGameSnapshot);
  wsManager.on('timeout_warning', () => {
    wx.showToast({ title: '已超时，系统将自动操作', icon: 'none' });
    audio.playTick();
  });
  wsManager.on('resource_update', onResourceUpdate);
  wsManager.on('error', onError);
  wsManager.on('opponent_disconnected', () => wx.showToast({ title: '对手已掉线，30秒后判你胜', icon: 'none' }));
}
function removeWs() {
  wsManager.off('stage_change', onStageChange);
  wsManager.off('piece_placed', onBoardUpdate);
  wsManager.off('capture_made', onBoardUpdate);
  wsManager.off('move_made', onBoardUpdate);
  wsManager.off('linked_capture', onLinkedCapture);
  wsManager.off('draw_rejected', onDrawRejected);
  wsManager.off('draw_request', onDrawRequest);
  wsManager.off('game_settle', onGameSettle);
  wsManager.off('game_snapshot', onGameSnapshot);
  wsManager.off('timeout_warning');
  wsManager.off('resource_update', onResourceUpdate);
  wsManager.off('opponent_disconnected');
  wsManager.off('error', onError);
}
function onBoardUpdate(data) {
  const remainingTime = data.remainingTime || 0;
  Object.assign(game, {
    currentTurn: data.currentTurn,
    remainingTime,
    timerText: gameCore.formatTime(remainingTime),
    timeLow: remainingTime > 0 && remainingTime <= 10,
  });
}

```

```

timerProgress: calcTimerProgress(remainingTime),
myTurn: data.currentTurn === game.myColor,
legalCells: [], selectedPieceIndex: -1, skipAvailable: false,
});
updateBoardPieces(data.board || []);
updateCatchNums(data.catchNums || { black: 0, white: 0 });
// 音效: 根据上一步操作类型播放 (lastMove.player 是执子方 color, action 是 place/move/capture)
const lm = data.lastMove;
if (lm && typeof lm.action === 'string') {
  if (lm.action === 'place') audio.playPlace();
  else if (lm.action === 'move') audio.playMove();
  // capture 由 updateCatchNums 内部的"我方被揪"逻辑播放 playCaptured+震动
}
}
function onStageChange(data) {
  const phase = gameCore.resolveStage(data.stage);
  Object.assign(game, {
    phase,
    phaseLabel: gameCore.STAGE_LABELS[phase] || game.phaseLabel,
    currentTurn: data.currentTurn,
    remainingTime: data.remainingTime || 0,
    timerText: gameCore.formatTime(data.remainingTime || 0),
    timeLow: (data.remainingTime || 0) > 0 && (data.remainingTime || 0) <= 10,
    myTurn: data.currentTurn === game.myColor,
    legalCells: [], selectedPieceIndex: -1, skipAvailable: false,
  });
  updateBoardPieces(data.board || []);
  updateCatchNums(data.catchNums || { black: 0, white: 0 });
}
function onLinkedCapture(data) {
  const remainingTime = data.remainingTime || 0;
  Object.assign(game, {
    currentTurn: data.currentTurn, remainingTime,
    timerText: gameCore.formatTime(remainingTime),
    timeLow: remainingTime > 0 && remainingTime <= 10,
    myTurn: data.currentTurn === game.myColor,
    legalCells: [], selectedPieceIndex: -1,
    skipAvailable: !!data.linkedCapture,
  });
  updateBoardPieces(data.board || []);
  updateCatchNums(data.catchNums || { black: 0, white: 0 });
}
function onDrawRejected(data) {
  wx.showToast({ title: '对方拒绝了求和', icon: 'none' });
  game.drawPaused = false;
  onBoardUpdate(data);
}

```

```

}
function onDrawRequest(data) {
  game.drawRequestBy = data.by;
  game.drawRequestName = data.nickname || '';
  game.showDrawRequest = true;
  game.drawPaused = true;
}
function onGameSettle(data) {
  // 只处理当前对局的结算, 忽略旧对局/其它对局的消息
  if (data && data.gameld && game.gameld && data.gameld !== game.gameld) {
    console.log('[Match] 收到非当前对局的结算消息, 已忽略:', data.gameld);
    return;
  }
  const myColor = game.myColor;
  let myResult = '';
  let scoreChange = 0;
  const endReason = data.endReason || '';
  if (data.result === 'black' && myColor === 1) { myResult = 'win'; scoreChange = data.blackRatingChange || 10; }
  else if (data.result === 'white' && myColor === 2) { myResult = 'win'; scoreChange = data.whiteRatingChange || 10; }
  else if (data.result === 'draw') { myResult = 'draw'; scoreChange = myColor === 1 ? (data.blackRatingChange || -1) : (data.whiteRatingChange || -1); }
  else { myResult = 'lose'; scoreChange = myColor === 1 ? (data.blackRatingChange || -3) : (data.whiteRatingChange || -3); }
  const rankKey = myColor === 1 ? 'blackNewRank' : 'whiteNewRank';
  const rankScoreKey = myColor === 1 ? 'blackAfterScore' : 'whiteAfterScore';
  Object.assign(game, {
    showSettle: true,
    settleData: Object.assign({}, data, { endReason }),
    myResult, scoreChange, myTurn: false, drawPaused: false,
    phase: 'settled', phaseLabel: '已结束',
    rankName: data[rankKey] || '', rankScore: data[rankScoreKey] || 0,
  });
  // 结算音效 + 震动反馈
  if (myResult === 'win') { audio.playWin(); audio.vibrate(30); }
  else if (myResult === 'draw') { audio.playDraw(); }
  else { audio.playLose(); audio.vibrate(15); }
}
function onGameSnapshot(data) {
  console.log('[Match] 收到游戏快照, 恢复对局状态');
  const payload = data && data.gameld ? data : state.currentGame;
  if (!payload) return;
  state.currentGame = payload;
  sceneMgr.goto('match', payload);
}
function onResourceUpdate(data) {
  if (data.energy !== undefined) state.energy.current = data.energy;
  if (data.energyRecoverAt !== undefined) state.energy.nextRecoverAt = data.energyRecoverAt;
}

```

```

if (data.energyMax !== undefined) state.energy.max = data.energyMax;
if (data.rankScore !== undefined) state.rankScore = data.rankScore;
if (data.rankName) state.rankName = data.rankName;
}
function onError(data) {
const msg = data && data.errMsg ? data.errMsg : '操作失败';
// 对局状态丢失时, 先尝试 reconnect 同步快照, 避免误弹窗
if (msg.indexOf('不在对局') >= 0 || msg.indexOf('未找到') >= 0) {
if (!game._reconnectTried) {
game._reconnectTried = true;
console.log('[Match] 收到未找到对局错误, 尝试 reconnect');
wsManager.send('reconnect', { gameId: game.gameId });
// 2 秒后若仍处于异常则提示返回
setTimeout(() => {
if (game.phase !== 'settled') {
wx.showModal({
title: '对局已结束',
content: '当前对局已不在进行中, 请返回大厅重新开始。',
showCancel: false,
success: () => sceneMgr.goto('home'),
});
}
}, 2000);
return;
}
wx.showModal({
title: '对局已结束',
content: '当前对局已不在进行中, 请返回大厅重新开始。',
showCancel: false,
success: () => sceneMgr.goto('home'),
});
return;
}
// 过滤良性/瞬时错误 (请先登录等重连竞态), 不打断正常游戏
if (msg === '请先登录' || /未知指令/.test(msg)) return;
wx.showToast({ title: msg, icon: 'none' });
}
// ===== 棋盘数据转换 =====
function updateBoardPieces(board) {
if (!board || !board.length) return;
const pieces = [];
const myColor = game.myColor;
for (let r = 0; r < 6; r++) {
for (let c = 0; c < 6; c++) {
const colorVal = board[r] ? board[r][c] : 0;
if (colorVal === 0) continue;
const color = colorVal === 1 ? 'black' : 'white';

```

```

pieces.push({ r, c, color, selected: false });
}
}
game.boardPieces = pieces;
game.board = board;
updateRemainCounts();
}
function updateRemainCounts() {
const board = game.board || [];
const { black, white } = gameCore.countPieces(board);
const myCount = game.myColor === 1 ? black : white;
const oppCount = game.myColor === 1 ? white : black;
game.myRemainText = myCount + '/18';
game.opponentRemainText = oppCount + '/18';
}
function updateCatchNums(catchNums) {
const myColor = game.myColor;
const prevOpp = game.opponentCatchNum || 0;
game.catchNums = catchNums;
game.myCatchNum = myColor === 1 ? (catchNums.black || 0) : (catchNums.white || 0);
game.opponentCatchNum = myColor === 1 ? (catchNums.white || 0) : (catchNums.black || 0);
// 我方棋子被揪走: 触发被揪音效 + 震动
if (game.opponentCatchNum > prevOpp) {
audio.playCaptured();
audio.vibrate(20);
}
updateCaptureMode();
}
function updateCaptureMode() {
const moveCaptureMode = game.phase === 'move' && game.myTurn && game.myCatchNum > 0;
if (moveCaptureMode !== game.moveCaptureMode) {
game.moveCaptureMode = moveCaptureMode;
game.legalCells = [];
game.selectedPieceIndex = -1;
}
}
// ===== 绘制 =====
function onDraw(ctx) {
// ctx.canvas.width 为物理尺寸, 需除以像素比得到逻辑尺寸
const pr = state.pixelRatio || 1;
W = ctx.canvas.width / pr;
H = ctx.canvas.height / pr;
// 本地倒计时自减: 保证环形进度和秒数字每秒实时变化
const nowTs = Date.now();

```